ECEN 260 - Final Project #1

Soda Storage System

Benjamin Marsh

Instructor: Brother Lynn Watson
Date: 4/8/2026
BYU-Idaho Semester: Winter 2026

Contents

1 Final Overview **3**
1.1 Objectives 3

2 Design **4**

3 Specifications **5**
3.1 Technical Details 5
3.2 Use Cases and Operating Instructions 5
3.3 Operating Constraints 6
3.4 Design Limitations 6
3.5 Parts List 6

4 Schematics **8**

5 Test Plan and Test Results **10**

6 Relevant Code **12**
6.1 Code selection from main while loop 12
6.2 Code selection from main.c 12
6.3 Code selection from ESP32Portion.ino 13

7 Visuals **14**

8 Conclusion **16**

List of Figures

1 This is a schematic of the working system. The ESP32 and NUCLEO boards are referenced alongside the SG90 mini servo motor and 1602 LCD I2C. . . . 8

2 This is an image of the pinout view STM32CubeMx will show once the project is configured. This pinout should be replicated before attempting to make changes in the Mode and Configuration of any settings. 9

3 A prototype of system circuit wiring installed in prototype box dispensing a soda. 15

1 Final Overview

This report is for the final project of ECEN 260 at Brigham Young University–Idaho. I created a Soda Storage System using a Nucleo STM32. This system stored soda (which from this point on will be called an *item*) and kept track of the stock of stored item quantities and changes to these quantities. This STM32 was connected to an ESP32 which controlled restocking a rudimentary user interface, and was connected via a full duplex Serial connection and a line dedicated to Falling Edge interrupts.

This system was connected to a slanted box that stored items with a servo motor acting as a latch at the end of the box, and had a Liquid Crystal Dispay (LCD) screen on top, which showed the current stock of the device. When a signal was received from the ESP32 (which simulated a payment processor) the STM32 would swing the latch open and closed. An item would roll freely out of storage before the latch re-engaged, and then the screen would display updated stock. The primary objectives for this project were therefore as follows:

1.1 Objectives

- Use Universal Asynchronous Receiver-Transmitter (UART) communication to communicate from one micro controller to another.
- Understand and use timers and Pulse Width Modulation (PWM) to use servo motors.
- Use an Inter-Integrated Circuit (I2C) LCD to display variables.
- Understand and use interrupts.

2 Design

My project fulfills the need for storage, release and tracking of on-demand items and beverages such as sodas.

In the creation of my program, I focused specifically on social factors, initially writing code for a communal storage and release system with a hosted webpage to enable decentralized connection to the system and pressing a button for the release of soda. Throughout the programming and system design process, this idea remained relatively consistent; however, an issue occurred when the ESP32 responsible for routing traffic, hosting a DNS and the webserver broke its radio antenna, resulting in a boot loop whenever I tried to run Wi-Fi-related code.

This changed my initial idea of a public storage or vending machine project into a more centralized storage and access system for my own use. Because of this, the project is likely to cause my soda consumption to increase. Considering public health and public welfare, the consumption of ultra-processed beverages is not optimal, so users of this soda storage system will face health consequences. Limits should be set to prevent users from drinking too much soda. Considering public safety, the device should only have one servo motor that moves 25° and then returns to rest. It is a pinch hazard. Pressurized cans of carbonated liquid can be dangerous if dropped. The servo should be located at an angle to optimally disperse these cans where it will not pinch fingers and the cans should have a safe place to rest because if punctured, they could depressurize and cause spills.

With global factors, cultural factors, social factors, and environmental factors in mind, carbonated beverages sold as cans are manufactured in the United States and most of the world. These beverages can only not be found in North Korea and Cuba, due to longstanding embargoes on these countries, and so the soda storage system is something that has wide use in the world at large. Culturally vending machines are used more in some countries than others, so most cultures would be familiar with the concept. Socially, this would be an interesting way to store cans, and considering economic factors, parts and pieces would not be too hard to buy in our current environment.

My project applies engineering design principles to integrate the following major concepts: Rising edge interrupts, which signal a purchase and prompts a display update, stock update, and servo movement, PWM to swing the servo back and forth, Timers to regulate the PWM, UART communication between my micro controller and an ESP32 which uses UART communication with a Laptop to handle restocking, and a Display (I2C communication) to display stock and when out of stock.

3 Specifications

This section should include all the specifications of the Soda Storage System. This means that it includes the technical details of the systems running on it such as PWM or UART and their configurations, alongside the specifications and configurations of each other system used by this device. Alongside this it should also include the use cases and operating procedures, operating constraints, design limitations and parts list of the system. Each relevant above-mentioned topic has a subsection below.

3.1 Technical Details

The following are technical details required to generate drivers with STM32CubeMX for the Nucleo32-L476RG. After setting the pins to match the specifications shown on the pinout schematic [2], make the following changes on the leftmost pane of STM32CubeMX:

- Navigate to GPIO. Under "Configuration" and "GPIO" set GPIO mode of PA10 to Eternal Interrupt with Falling edge trigger detection. Then under "NVIC" enable EXTI line 15:10 interrupts.
- Navigate to TIM2. Set TIM2 to enabled with Clock source of Internal Clock. Set Channel 1 of TIM2 to PWM Generation CH1. Under "Configuration" and "Parameter Settings", set the prescaler to 39, the counter period to 39999, and auto-reload preload set to Enable.
- Navigate to I2C1. Change I2C1 from disabled to I2C. Under "Configuration" and "GPIO Settings" set both PB8 and PB9 to Pull-up.
- Navigate to UART4. On UART4 change mode from disabled to asynchronous. Under "Configuration" and "Parameter Settings" set the Baud Rate to 9600. Under "NCIV Settings" enable the UART4 global interrupt.
- Navigate to NVIC settings and ensure that in "NVIC" EXTI line[15:10] interrupts and UART4 global interrupts are enabled.

Once finished, save the project and generate the drivers in the STM32CubeIDE. Load the code from below into the main.c file and build it onto the STM32 board. Load the code for the ESP32 into the Arduino IDE and build it onto the ESP32 board.

The STM32CubeMX project folder used to make the drivers I used can also be found online at `https://github.com/Serverror7/SodaStorageSystem`.

3.2 Use Cases and Operating Instructions

See Figure 1 for the schematic of the wiring for this lab.

- Once the circuit is assembled, and code running on the ESP32 and STM32 ensure that the LCD screen is secured to the top of the box and the servo latch is in a position where it will block and then release soda.

- Plug the STM32 into power and the ESP32 into a laptop with a serial terminal (such as PuTTY) open.
- Readjust the servo motor latch if necessary (it will spin to a resting position once on).
- Place stock inside the box in a position so that the items will roll down into the release area when the latch opens.
- On the laptop serial terminal connected to the ESP32 enter the number of items added and hit enter (This is written numerically, ergo "20", not "twenty"). This number should appear on the LCD.
- On this same terminal enter a non-numeric character and hit enter (a non-numeric character is a character such as "k" and not a number 0 through 9). The latch should release and then reset and the number of stock will decrement as an item rolls from the storage system.

3.3 Operating Constraints

- The ESP32 connects with a laptop on a 115200 baud serial connection.
- If the connection from the ESP32 to the STM32 or to a Laptop is not secure (ie fastened), stock cannot be updated on the ESP32.
- If the connection from the ESP32 to the STM32 or to a Laptop is not secure payment cannot be read on the payment line or faulty payments will occur.
- The servo latch spins counterclockwise then clockwise. When installed.

3.4 Design Limitations

- While the ESP32 does emulate a payment processor, it is not a real payment processor.
- UART protocol is used for the ESP32 and the STM32 to communicate and send some data, however the STM32 does not use its RX pin to send messages to the ESP32.
- The item release mechanism and box design only works if the items are cylindrical and roll due to gravity.

3.5 Parts List

- STM32 micro controller specifically the Nucleo32-L476RG.
- ESP32 micro controller specifically the ESP32 Development Board.
- A SG90 micro servo motor.
- A 1602 Character LCD with I2C daughterboard

- Wires.
- A computer with a serial terminal and a micro USB cable to connect said computer with ESP32

"For specifics on the wiring setup, see the schematics in Section 4."

4 Schematics

The following are schematics used to recreate the Soda Storage System:

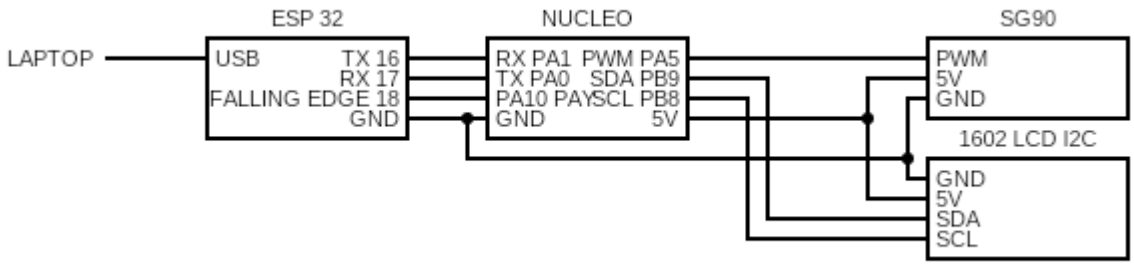

Figure 1: This is a schematic of the working system. The ESP32 and NUCLEO boards are referenced alongside the SG90 mini servo motor and 1602 LCD I2C.

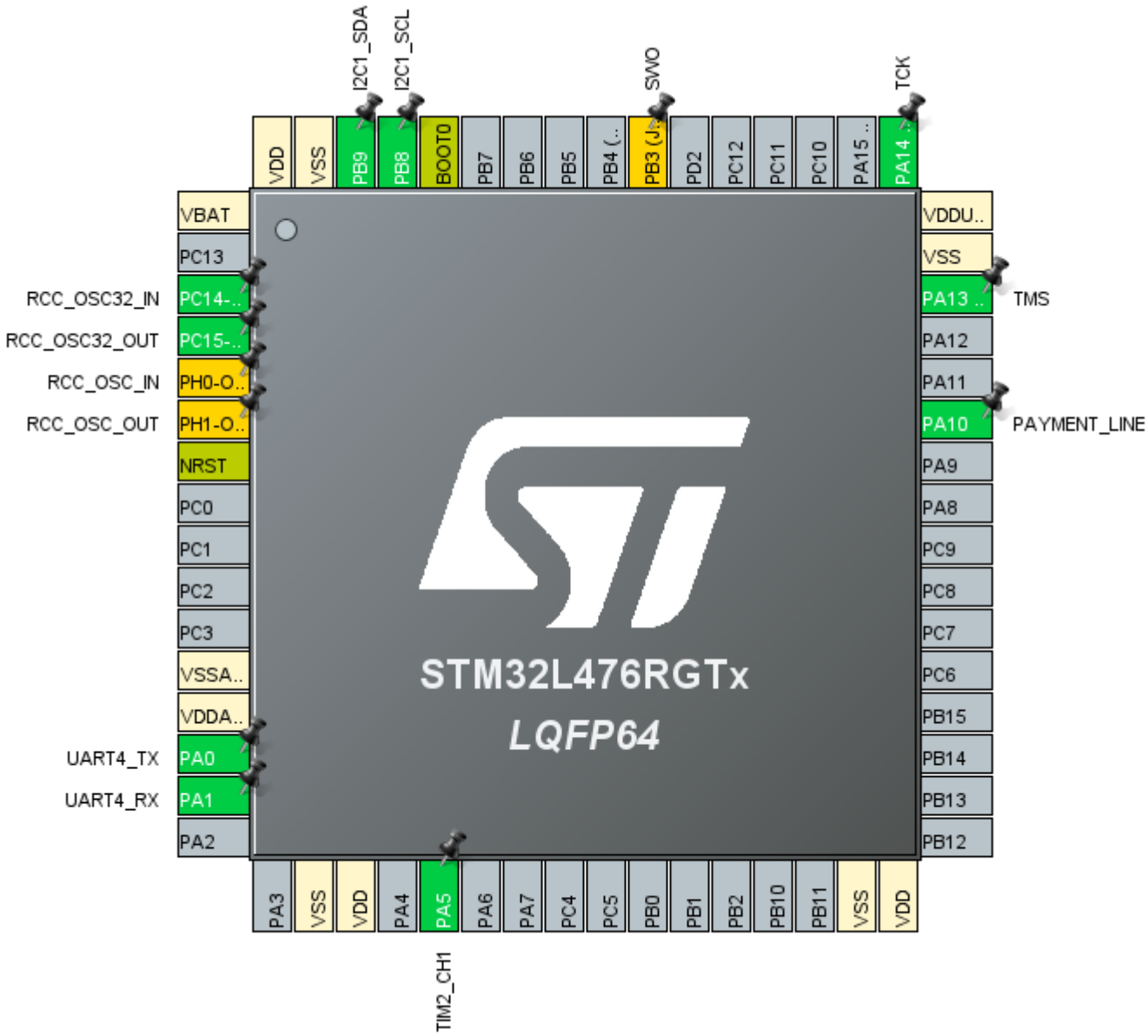

Figure 2: This is an image of the pinout view STM32CubeMx will show once the project is configured. This pinout should be replicated before attempting to make changes in the Mode and Configuration of any settings.

5 Test Plan and Test Results

A video demo demonstrating the use of the Soda Storage System can be found here. Test cases include the following:

1. The machine has no items in it and its stock count is 0 and the user tries to receive an item.
2. The machine has items in it but its stock count is 0 and the user tries to receive an item.
3. The machine has one stock count and receives another different stock count.
4. The machine has a stock count equal to the number of items inside and the user tries to receive an item.
5. The machine has stock count greater than 0 and there are no items inside.

- Test Scenario #1 - The machine has no items in it and its stock count is 0 and the user tries to receive an item.
  - Step 1: In terminal set stock to 0, or restart STM32 and ensure stock is set to 0.
  - Step 2: In terminal type a non-numeric character and hit enter.
  - Expected result: Watch for OUT OF STOCK to flash on the screen for a second, then disappear.
  - Actual result: As expected.
- Test Scenario #2 - The machine has items in it but its stock count is 0 and the user tries to receive an item.
  - Step 1: Place items in the storage system.
  - Step 2: In terminal set stock to 0, or restart STM32 and ensure stock is set to 0.
  - Step 3: In terminal type a non-numeric character and hit enter.
  - Expected result: Watch for OUT OF STOCK to flash on the screen for a second, then disappear.
  - Actual result: As expected.
- Test Scenario #3 - The machine has one stock count and receives another different stock count.
  - Step 1: In terminal set stock to 6.
  - Step 2: In terminal set stock to 7.
  - Expected result: The stock counter on the LCD should read 7.
  - Actual result: As expected.

- Test Scenario #4 - The machine has a stock count equal to the number of items inside and the user tries to receive an item.
  - Step 1: Place items in the storage system.
  - Step 2: In terminal set stock to be equal to number of items in storage.
  - Step 3: In terminal type a non-numeric character and hit enter.
  - Expected result: Watch for latch to move, can to roll out, latch to reset and then stock on screen to decrement.
  - Actual result: As expected.
- Test Scenario #5 - The machine has stock count greater than 0 and there are no items inside.
  - Step 1: Remove all items from the storage system.
  - Step 2: In terminal set stock to be greater than 0.
  - Step 3: In terminal type a non-numeric character and hit enter.
  - Expected result: Watch for latch to move and reset and then stock on screen to decrement.
  - Actual result: As expected.

6 Relevant Code

These are relevant selections of code from the project. All code is uploaded to github and can be found here: `https://github.com/Serverror7/SodaStorageSystem`

6.1 Code selection from main while loop

```c
volatile uint8_t payment_received = 0; // Start at 0 because this turns on when PA10 detects a falling edge.
volatile int current_stock = 0; // Start at 0 stock. Can be configured.
volatile uint8_t update_display_flag = 1; // Start at 1 to show initial stock, will turn to 0 after screen update.
int main(void)
{
  while (1) {
      if (payment_received) { // if payment received is 1
          if (current_stock > 0) { // and the current stock is greater than 0
              current_stock--; // decrement current stock
              Servo_Swing_And_Return(25.0); // release latch
              update_display_flag = 1; // set display flag to 1
          } else { // if payment is recieved and stock is 0 or less
              CharLCD_Set_Cursor(1, 0); // move cursor to second row
              CharLCD_Write_String("OUT OF STOCK!   "); // Rewrite over stock counter
              HAL_Delay(1000); // Wait a second
              update_display_flag = 1; // Set flag to 1 to trigger reprinting of stock.
          }
          payment_received = 0; // Reset payment flag to 0 will trigger again with interrupt.
      }

      if (update_display_flag) { // if display flag is 1
          char buffer[16]; // create a buffer to store data in
          sprintf(buffer, "Stock: %d       ", current_stock); // add stock text into buffer.
          CharLCD_Set_Cursor(1, 0); // move cursor
          CharLCD_Write_String(buffer); // print buffer onto second line
          update_display_flag = 0;
      }
  }
}
```

6.2 Code selection from main.c

```c
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) { // On a falling edge
    if (GPIO_Pin == PAYMENT_LINE_Pin) { // If PA10 triggered it
        payment_received = 1; // Set payment flag to 1
    }
}
```

6.3 Code selection from ESP32Portion.ino

```
void setup() {
  Serial.begin(115200); // Set up serial connection with laptop
  Serial2.begin(9600, SERIAL_8N1, 16, 17); // Set up serial connection with STM32

  pinMode(TRIGGER_PIN, OUTPUT); // Set TRIGGER_PIN (18) to be an output
  digitalWrite(TRIGGER_PIN, HIGH); // Set TRIGGER_PIN to be high

  Serial.println("Ready. Enter a number to update stock, or any key for a trigger pulse."); // Print to laptop when on.
}

void loop() {
  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    input.trim(); // cut input

    if (input.length() > 0) {
      if (isNumeric(input)) { // if it's a number send stock to Nucleo
        Serial2.println(input); // print the number to STM32
        Serial.print("Stock Sent: "); // print confirmation of sending to laptop
        Serial.println(input);
      } else { // if its not a number pulse GPIO 18
        digitalWrite(TRIGGER_PIN, LOW); // trigger falling edge
        delay(10); // wait
        digitalWrite(TRIGGER_PIN, HIGH); // rise it
        Serial.println("Non-numeric input detected: Falling edge triggered on GPIO 18."); // print confirmation to laptop.
      }
    }
  }
}
```

7 Visuals

A video of this system working can be found at `https://youtube.com/shorts/5umbk6tQL0I?feature=share`.

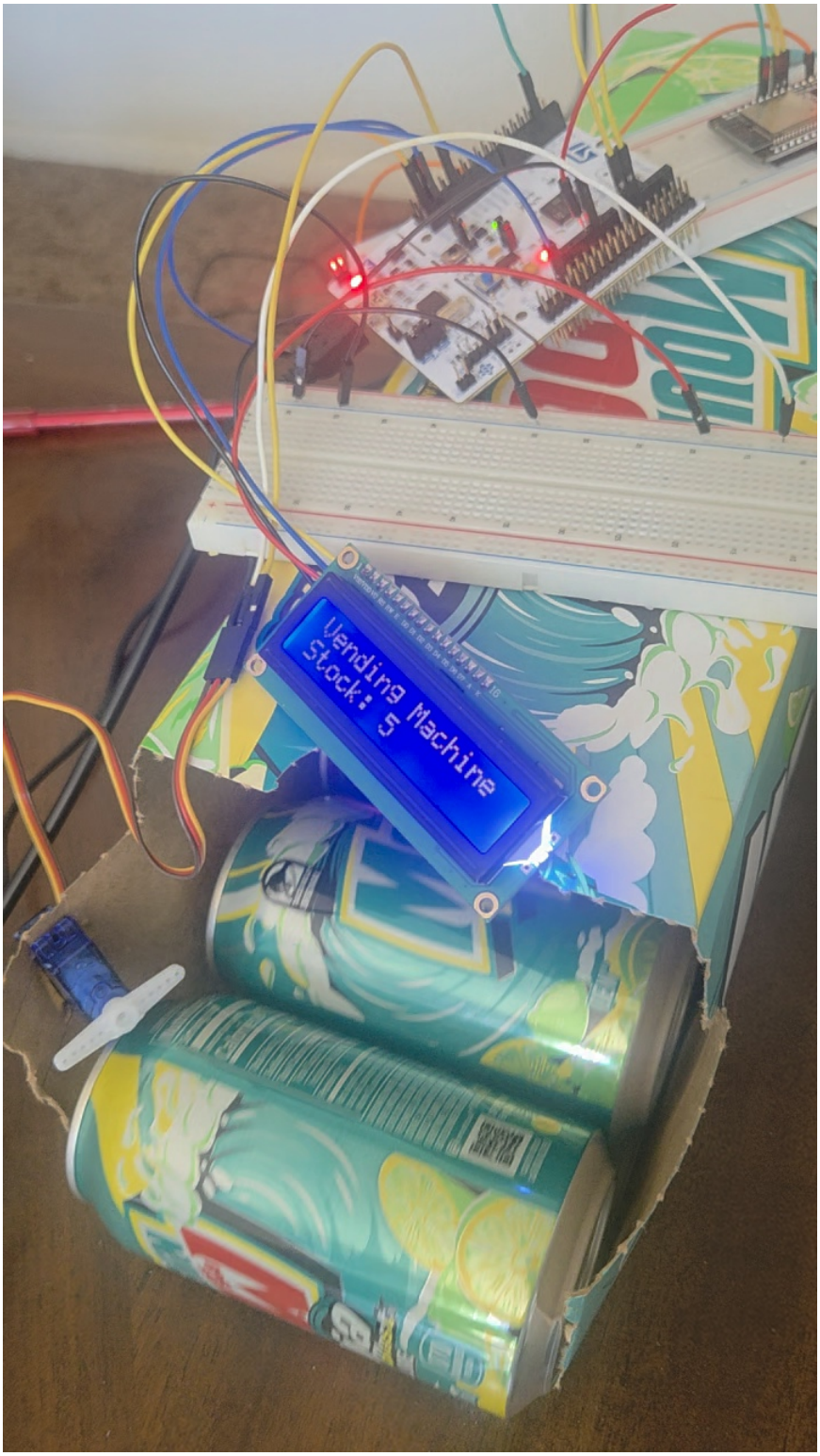

Figure 3: A prototype of system circuit wiring installed in prototype box dispensing a soda.

8 Conclusion

Making this project, I worked with the STM32 to understand UART communication, I2C, timers and PWM, and interrupts. I also worked with the ESP32 in order to simulate a second micro controller. Working with a servo motor, LCD and another micro controller, I designed and made a system that works within the specifications I decided for this lab.

Micro controllers, I2C devices, and PWM are all resources that I will need to take advantage of in the future, especially if I want to make more advanced systems and devices. I loved working with the ESP32 and I am sad that I broke mine, however I think I will buy another in the future. I enjoyed this project and want to make more projects like this but with Wi-Fi or Bluetooth capabilities. I want to work with more micro controllers and devices such as these in my future carreer.

References